Proceedings of the 10th International and 50th National Conference on Fluid Mechanics and Fluid Power (FMFP)
December 20–22, 2023, IIT Jodhpur, Rajasthan, India

FMFP2023-XX-YYY

GPU-Accelerated Implementation of the Savage–Hutter Model with Application to 1D and 2D Inclined Plane Granular Flows

Samyak Mayank[1]

[1]School of Mechanical and Materials Engineering, Indian Institute of Technology Mandi, Himachal Pradesh 175075, India

ABSTRACT

Granular flows such as rockfalls, landslides, and snow avalanches pose serious hazards in mountainous regions and are characterized by complex rheology, intermediate between solid-like and fluid-like behavior. This paper models the motion of a finite mass of cohesionless granular material released from rest on a rough inclined plane using the depth-averaged Savage–Hutter equations. Numerical simulations are validated against analytical similarity solutions (parabolic cap profile) and then accelerated using CUDA-based GPU parallelization. A performance comparison between Python, C++ (CPU), and CUDA (GPU) implementations highlights the computational advantages of GPU computing for large-scale granular flow simulations. The framework is further extended to a full two-dimensional finite-volume Rusanov solver, rigorously validated against the TITAN2D reference code using Intersection-over-Union (IoU), RMSE, and Lagrangian centroid tracking metrics.

Keywords: Granular Avalanches, Savage–Hutter Model, GPU Acceleration, CUDA, Finite Difference Method, Finite Volume Method

I. Introduction

Granular flows such as rockfalls, landslides, and snow avalanches constitute major geological hazards in mountainous regions. Unlike simple Newtonian fluids, granular materials exhibit dual behavior: they can flow like fluids when mobilized, yet they also possess solid-like frictional strength that allows them to remain at rest on steep slopes. This combination of frictional resistance, dilatancy, and collisional interactions makes granular avalanche modeling non-trivial.

In order to investigate the fundamentals of avalanche motion, it is useful to focus on an idealized configuration: a finite mass of cohesionless granular material released from rest on a rough inclined plane. After release, the mass accelerates downslope under gravity, spreads due to internal pressure gradients, and eventually comes to rest as basal friction balances the driving component of gravity. The objective of this paper is threefold:

- To model the avalanche using depth-averaged continuum equations based on the Savage–Hutter framework.
- To implement a numerical finite-difference scheme and validate the simulation against the analytical parabolic cap similarity solution.
- To introduce GPU acceleration using CUDA and demonstrate the performance advantage of GPU-based implementation over Python and C++ CPU-based codes.

II. Literature Review

A. Historical Modeling Approaches

Early modeling of granular avalanches was largely based on concepts from static soil mechanics, especially the Mohr–Coulomb yield criterion. These approaches are suitable for predicting the onset of failure and limiting equilibrium, but they offer limited insight into the dynamic evolution of a moving granular mass once it is in motion. In the context of snow avalanches, hydraulic analogies led to semi-empirical models such as those by Voellmy and Salm, which treated the avalanche as a flowing fluid with additional friction and turbulence-like terms. While useful for engineering applications, these models often lacked a rigorous derivation for the dynamics of a finite mass of granular material with evolving free boundaries.

B. Granular Flow Theory

With the development of granular flow physics, kinetic theory-inspired models emerged to describe rapid flows dominated by grain collisions. These models account for rate-dependent stresses, velocity fluctuations and granular temperature, and collisional transport of momentum and energy. However, many natural avalanches operate in a dense flow regime where enduring contacts, frictional interactions, and occasional collisions all play a role. This intermediate regime is more difficult to model analytically, and fully kinetic-theory-based models become complex and computationally expensive.

C. The Savage–Hutter Approach

The Savage–Hutter model bridges the gap between purely static and fully collisional descriptions by treating the avalanche as an incompressible Coulomb-like continuum. The key features are: depth-averaging of the mass and momentum equations under a shallow-flow assumption; a Coulomb-type basal friction law with constant friction angle; and incorporation of internal friction via earth-pressure coefficients relating normal and shear stresses within the flow. The resulting system structurally resembles the nonlinear shallow-water equations, but with modifications that reflect granular frictional behavior rather than fluid viscosity.

D. Experimental Validation

Laboratory experiments on gravel and granular materials released down inclined chutes provide crucial validation data. Measurements of the front and rear margins as a function of time show that the Savage–Hutter depth-averaged model captures the spreading and flattening of the pile, the approximate runout distances, and the tendency of the profile to evolve towards specific self-similar shapes (parabolic cap).

These experiments support the use of the Savage–Hutter framework as a physically grounded yet computationally tractable model for granular avalanches.

III. Problem Definition

A. Physical Setup

We consider a finite mass of cohesionless granular material initially at rest on a rigid, planar bed inclined at an angle ζ to the horizontal. The granular body has an initial thickness distribution $h(x, 0)$, nonzero over a finite interval, and zero elsewhere. When the material is released, gravity accelerates the mass downslope, internal pressure gradients drive spreading, and basal friction decelerates and eventually stops the flow. The free surface and the positions of the front and rear margins evolve in time, making this a moving-boundary problem.

B. Modeling Assumptions

a) Incompressibility.: The bulk density ρ is taken to be constant and changes in void ratio during flow are neglected, so mass conservation simplifies to depth conservation.

b) Shallow Flow.: The flow is assumed to be long and shallow, with a small aspect ratio

$$\epsilon = \frac{H}{L} \ll 1,$$

where H is a characteristic thickness and L a characteristic length along the slope. This justifies using a hydrostatic normal stress approximation and performing depth-averaging.

c) Depth Averaging.: Velocity and stress variations in the vertical direction are represented through their depth-averaged counterparts. The primary kinematic variables are therefore $h(x, t)$ (flow thickness) and $u(x, t)$ (depth-averaged downslope velocity).

d) Frictional Resistance.: Basal friction is modeled by a Coulomb sliding law with constant basal friction angle δ: $\tau_b = \sigma_n \tan\delta$, where τ_b is the basal shear stress and σ_n the normal stress at the bed. Internal friction is characterized by an angle ϕ, which enters the earth-pressure coefficient.

IV. Mathematical Formulation

Under the above assumptions and after non-dimensionalization, the depth-averaged governing equations for flow over a planar bed take the following form.

A. Conservation of Mass

$$\frac{\partial h}{\partial t} + \frac{\partial}{\partial x}(h\,u) = 0, \tag{1}$$

which expresses conservation of mass (or thickness) for the deforming granular body.

B. Conservation of Momentum

$$\frac{\partial u}{\partial t} + u\,\frac{\partial u}{\partial x} = \sin\zeta - \tan\delta\cos\zeta - \beta\,\frac{\partial h}{\partial x}, \tag{2}$$

where $\sin\zeta$ is the driving downslope gravity component, $\tan\delta\cos\zeta$ is the resisting basal Coulomb friction (for $u > 0$), and $\beta\,\partial h/\partial x$ is an effective pressure-gradient term. The coefficient

$$\beta = \epsilon\,k_{\text{act}}\cos\zeta, \tag{3}$$

where ϵ is the aspect ratio and k_{act} is the active earth-pressure coefficient.

C. Earth-Pressure Coefficient

The earth-pressure coefficient k_{act} is derived from the Mohr–Coulomb yield criterion under an active stress state (material effectively extending downslope). It depends primarily on the internal friction angle ϕ and slope angle ζ, and is treated as a constant for a given material and slope.

D. Boundary Conditions

Because the granular mass is finite, the thickness vanishes at the front and rear margins:

$$h(x_F(t), t) = 0, \qquad h(x_R(t), t) = 0, \tag{4}$$

with margins moving at the local flow velocity $dx_{F,R}/dt = u(x_{F,R}, t)$. In numerical simulations over a fixed domain $[L_{\text{left}}, L_{\text{right}}]$, we impose $h = 0$, $u = 0$ at both far-field boundaries.

V. Solution Techniques

A. Analytical Similarity Solutions

For special classes of initial conditions and under symmetry assumptions, the governing equations admit analytical self-similar solutions, which serve as benchmarks for validating the numerical implementation.

1) Parabolic Cap Solution: A particularly important similarity solution is the *parabolic cap*. In a suitably scaled comoving coordinate $\eta = (x - x_c(t))/g(t)$, where $x_c(t)$ is the center of mass and $g(t)$ is the half-width of the pile, the thickness profile takes the form

$$h(\eta, t) = \frac{K}{g(t)}\big(1 - \eta^2\big), \tag{5}$$

where K is a constant related to the conserved total mass. The depth-averaged velocity varies approximately linearly with space, $\tilde{u}(\eta, t) \propto \eta$, with maximum magnitude near the center and zero at the margins. The half-width grows asymptotically as $g(t) \sim t$ as $t \to \infty$, while the maximum height decays due to spreading.

B. Numerical Methods

For general initial conditions, the system (1)–(2) must be solved numerically. The computational domain $[L_{\text{left}}, L_{\text{right}}]$ is discretized into N_x grid points x_i, and time is advanced using an explicit forward Euler scheme:

$$q_i^{n+1} = q_i^n + \Delta t\,\mathcal{R}_i(q^n), \tag{6}$$

where $q = (h, u)$ and $\mathcal{R}$ denotes the discrete right-hand side. Stability requires a CFL condition relating Δt to mesh size and characteristic wave speeds. The convective term $u\,\partial u/\partial x$ is approximated using an upwind finite-difference scheme for stability, while $\partial h/\partial x$ uses centered differences to avoid artificial bias in the spreading direction. At the far-field boundaries, $h = 0$ and $u = 0$ are imposed directly.

VI. Simulation Scenarios and Validation Strategy

Three simulation cases are used to systematically verify the numerical solver, each isolating a specific part of the coupled mass–momentum system.

A. Case 1: Numerical Thickness, Analytical Velocity

The analytical similarity velocity profile is prescribed while the mass conservation equation is solved numerically to obtain $h(x,t)$ (Fig. 1). This validates the spatial discretization of the mass equation and confirms correct spreading dynamics and mass preservation.

B. Case 2: Numerical Velocity, Analytical Thickness

The analytical thickness profile is imposed while the momentum equation is solved numerically for $u(x,t)$ (Fig. 2). This evaluates the accuracy of the momentum solver and tests advection stability and internal-pressure effects independently.

C. Case 3: Fully Coupled Numerical Solution

Both mass and momentum equations are solved simultaneously using the finite difference scheme (Fig. 3), representing the full physical problem and assessing coupling stability and convergence to similarity form.

This staged validation structure gives strong evidence that all solver components are correct before GPU acceleration is introduced.

VII. Results and Discussion

The results demonstrate that the finite difference solver successfully reproduces the spreading and flattening behavior predicted by the Savage–Hutter model.

A. Case 1: Mass Conservation Verification

The numerical thickness profiles quickly relax into a parabolic shape. The front and rear margins advance outward at nearly constant rates for large t, matching the asymptotic spreading trend $g(t) \sim t$. Small deviations near boundaries are due to numerical diffusion inherent to first-order upwinding.

B. Case 2: Momentum Solver Verification

The numerically computed velocity distribution maintains nearly linear spatial variation $u(x,t) \propto \eta$, as predicted by the similarity solution. Minor phase lag between numerical and analytical profiles decreases with grid refinement, confirming correct implementation of internal-pressure and advective transport terms.

C. Case 3: Fully Coupled Dynamics

In the coupled case, the solution converges naturally toward the parabolic cap profile even when initial conditions are not perfectly parabolic, providing numerical confirmation of the theoretical *shape stability* of the solution. The spreading rate and profile alignment with analytical similarity solutions indicate strong accuracy of the coupled solver.

Algorithm 1: Serial CPU Execution

```
Allocate memory for state vector U = (h, hu, hv);
for t = 0 to T_end do
    Compute maximum wave speed and Δt;
    for j = 1 to N_y do
        for i = 1 to N_x do
            Compute k(x, y) using velocity
              divergence;
            Compute interface fluxes F̂_{i+1/2,j} and
              Ĝ_{i,j+1/2};
        end
    end
    for j = 1 to N_y do
        for i = 1 to N_x do
            Update state U^{n+1}_{i,j} via Eq. (21);
            Enforce dry-bed constraint (h ≥ 0);
        end
    end
end
```

VIII. GPU Programming and Performance

A. Motivation for GPU Acceleration

As spatial resolution and simulation time increase, the computational cost of solving the Savage–Hutter equations on a CPU becomes prohibitive. GPUs are designed for massively parallel computations and can execute thousands of lightweight threads simultaneously, making them well-suited for updating large arrays of h and u in parallel, applying stencil-based finite-difference operations, and running many time steps efficiently.

B. Methodology of GPU Acceleration

1) Conceptual Overview: In a standard CPU (serial) implementation, a single processing unit iterates through the grid cell-by-cell, computing inter-cell fluxes and updating each cell sequentially. As grid resolution scales to hundreds of thousands of cells, this sequential processing becomes a profound bottleneck. GPU acceleration takes a fundamentally different approach: the GPU deploys one thread per grid cell, and at each time step all threads simultaneously read their neighbors, compute fluxes, and update their respective cells concurrently.

2) Algorithmic Framework: Serial vs. Parallel: The transition from a sequential CPU model to a parallel CUDA model involves flattening the nested spatial loops. The explicit nature of the Rusanov scheme ensures that the update for cell (i,j) depends only on its neighbors at the current time step t^n, making the algorithm *embarrassingly parallel*.

3) Spatial Mapping and Kernel Execution: The two-dimensional computational domain is partitioned into a uniform grid of CUDA thread blocks (e.g., 16×16 threads per block). Physics computations are distributed across distinct `__global__` kernel functions to enforce global memory synchronization before advancing to subsequent operations. Inside each kernel, a thread determines its unique global

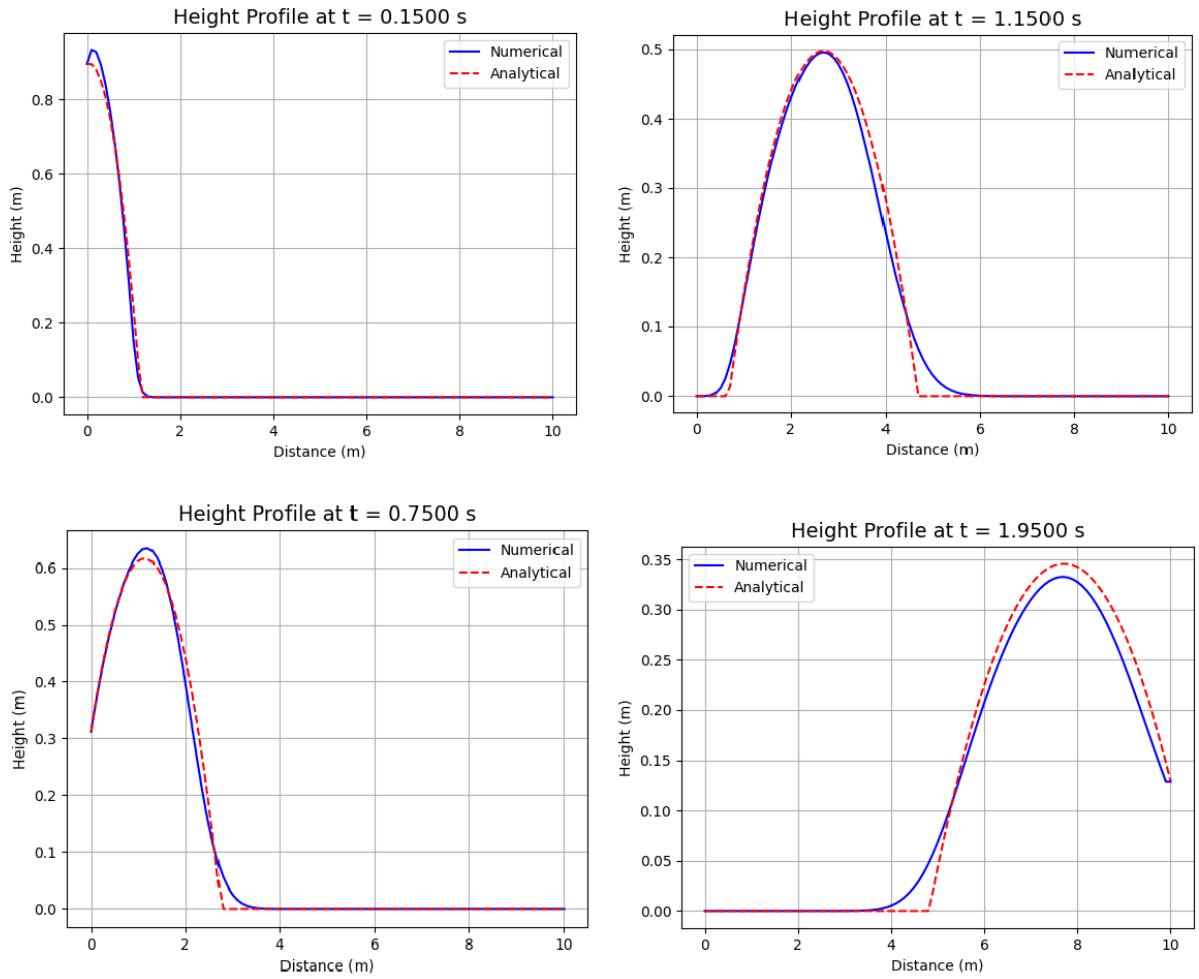

Figure 1: **Case 1: Evolution of flow thickness $h(x,t)$ using the prescribed analytical velocity profile. Numerical (solid) and analytical (dashed) solutions are shown at successive time instants.**

spatial indices:

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}, \quad (7)$$

$$j = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}. \quad (8)$$

4) Data Layout and Memory Optimization: Traditional object-oriented programming groups variables into an Array of Structures (AoS). The GPU implementation instead uses a Struct of Arrays (SoA), storing all flow thicknesses h contiguously in memory, followed separately by momentum arrays hu and hv. When a CUDA warp (32 synchronized threads) accesses SoA memory, it performs a single coalesced memory transaction, saturating the high-bandwidth GDDR memory bus. Host-to-device transfers (`cudaMemcpy`) occur only at initialization and specific checkpoints for data logging, eliminating PCIe bus latency during the temporal loop.

C. Performance Comparison and Analysis

The numerical solver was executed for grid resolutions from $N_x = 101$ to $N_x = 100{,}001$, measuring execution time for Python (NumPy-based), C++ (CPU), and CUDA (GPU) implementations.

As shown in Fig. 4, Python performs slowest across all grid sizes. The C++ CPU implementation scales far more efficiently and is significantly faster for small to medium grid sizes ($N_x < 25{,}000$). The GPU initially exhibits larger runtimes for smaller grids because kernel launch latency and host–device memory transfers dominate. However, as N_x increases, the advantage of massive parallelism becomes evident (Figs. 6 and 7). The GPU achieves more than $70\times$ speedup over Python at $N_x = 1601$, and outperforms C++ for grid sizes exceeding approximately $N_x = 25{,}601$. Compiler optimization levels for 2D grids are benchmarked in Fig. 5, with `-O2` and `-O3` achieving the highest gains over the unoptimized baseline. These results confirm that CUDA acceleration enables simulations at resolutions that would be impractical on CPU-based solvers alone.

Figure 2: **Case 2: Evolution of depth-averaged velocity $u(x,t)$ using the prescribed analytical thickness profile. Numerical (solid) and analytical (dashed) solutions are shown at successive time instants.**

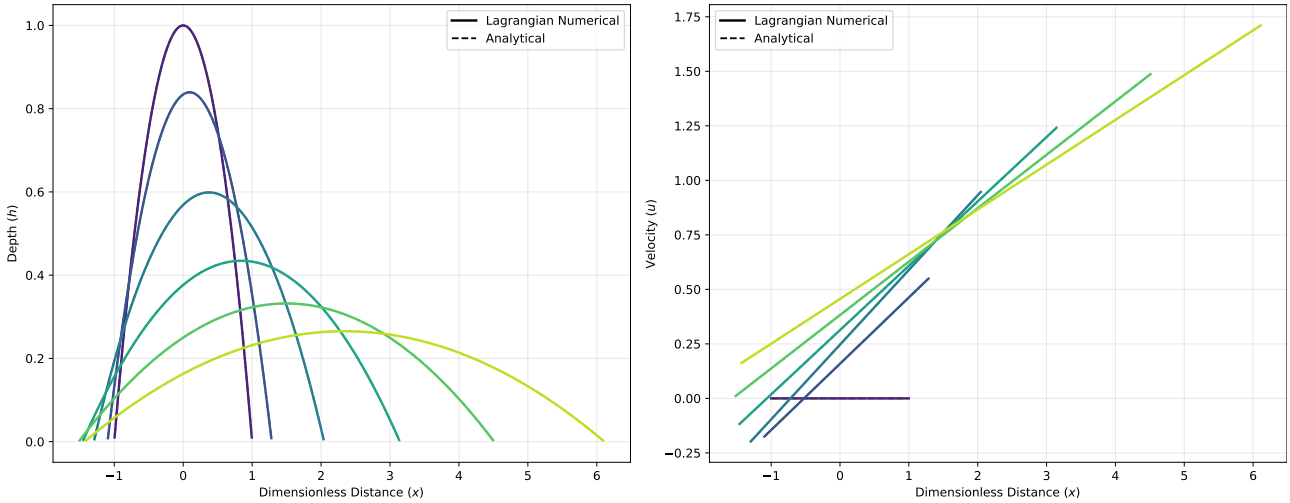

Figure 3: **Case 3: Fully coupled numerical profiles of $h(x,t)$ (left) and $u(x,t)$ (right) compared with the analytical Savage–Hutter parabolic cap similarity solution ($\zeta = 32°$, $\delta = 22°$, $\beta = 0.204$).**

IX. Extension to Two-Dimensional Flow

The one-dimensional framework captures the essential physics of depth-averaged granular flow but restricts spreading to a single downslope direction. Real avalanche events spread laterally as well, driven by internal pressure gradients in both the downslope (x) and cross-slope (y) directions. This section derives the two-dimensional governing equations, introduces the dynamic earth-pressure model, and describes the finite-volume Rusanov scheme used for their numerical solution.

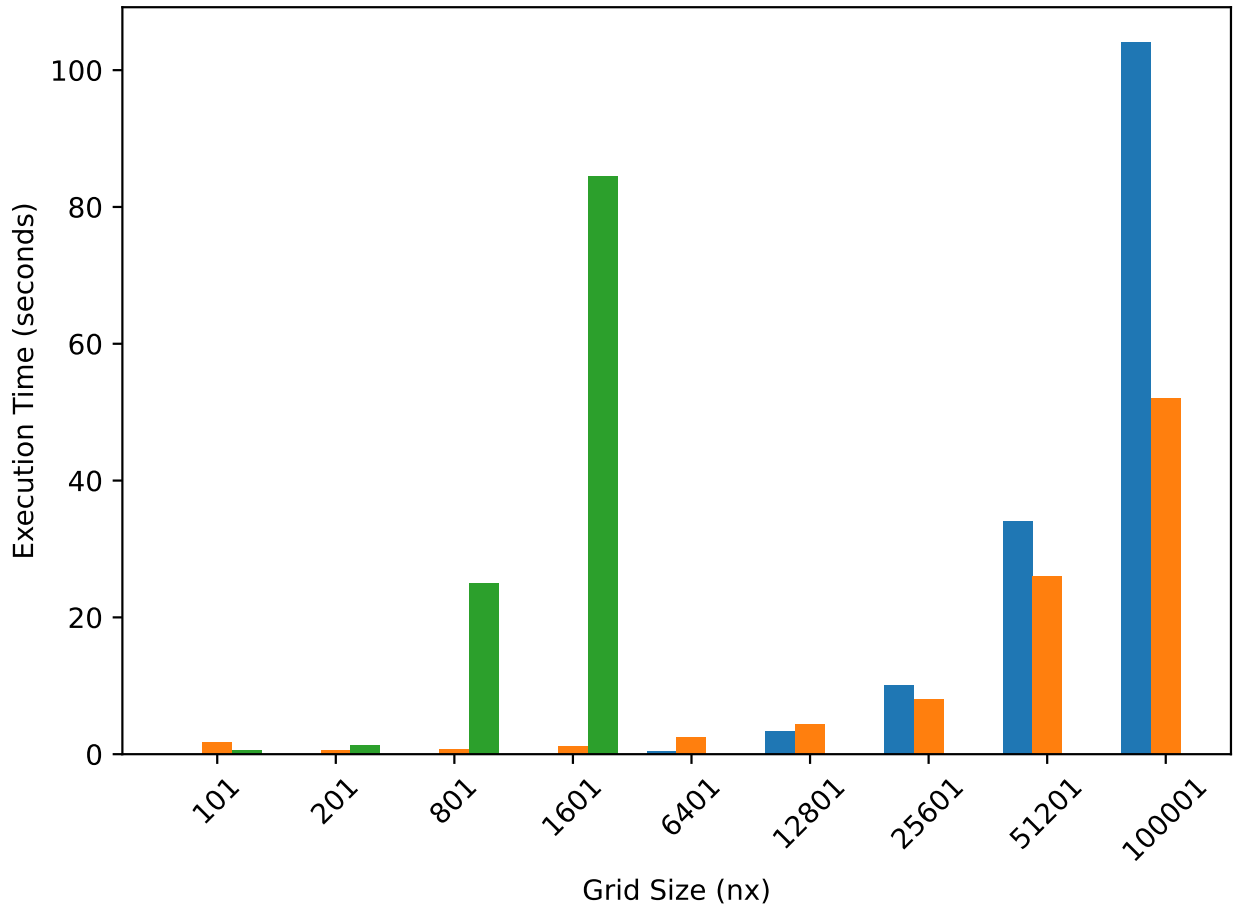

Figure 4: **Execution time for Python (green), C++ (orange), and CUDA (blue) implementations across different 1D grid sizes N_x. Python is slowest due to its interpreted nature; the GPU incurs overhead for small grids but dominates at large N_x.**

A. Two-Dimensional Governing Equations

1) Coordinate System: A bed-fitted orthogonal coordinate system is adopted: x is the downslope direction, y the cross-slope direction, and z normal to the inclined plane. The gravitational components resolved along these axes are

$$g_x = g\sin\zeta, \qquad g_y = 0, \qquad g_z = g\cos\zeta. \tag{9}$$

2) Conservation of Mass: Depth-averaging the three-dimensional incompressibility condition over the granular layer yields the two-dimensional continuity equation:

$$\frac{\partial h}{\partial t} + \frac{\partial (hu)}{\partial x} + \frac{\partial (hv)}{\partial y} = 0, \tag{10}$$

where $h(x, y, t)$ is the local flow thickness and $u(x, y, t)$, $v(x, y, t)$ are the depth-averaged velocity components in the x- and y-directions, respectively.

3) Conservation of Momentum: The depth-averaged momentum equations in conservative form read:

$$\frac{\partial (hu)}{\partial t} + \frac{\partial}{\partial x}\left(hu^2 + \tfrac{1}{2}kg_z h^2\right) + \frac{\partial (huv)}{\partial y} = S_x, \tag{11}$$

$$\frac{\partial (hv)}{\partial t} + \frac{\partial (huv)}{\partial x} + \frac{\partial}{\partial y}\left(hv^2 + \tfrac{1}{2}kg_z h^2\right) = S_y, \tag{12}$$

where $\frac{1}{2}kg_z h^2$ represents depth-averaged normal stress contributions scaled by the earth-pressure coefficient k.

4) Source Terms: The source terms S_x and S_y combine gravitational forcing, basal Coulomb friction, and internal cross-stress:

$$\begin{aligned} S_x = g_x h &- \frac{u}{|\boldsymbol{u}|}\, h g_z \mu \\ &- hk\,\mathrm{sgn}\left(\frac{\partial u}{\partial y}\right)\sin\phi\,\frac{\partial (g_z h)}{\partial y}, \end{aligned} \tag{13}$$

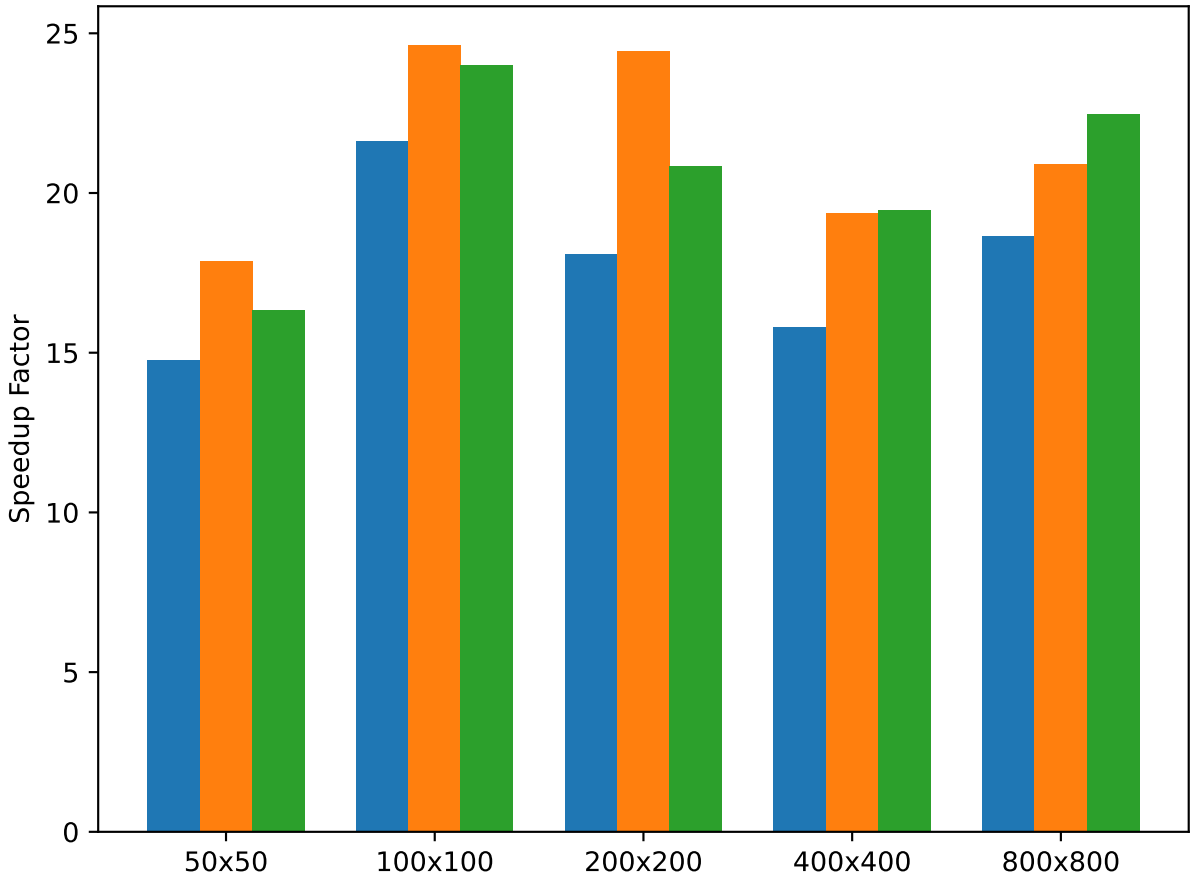

Figure 5: **Speedup achieved by compiler optimization levels `-O1` (blue), `-O2` (orange), and `-O3` (green) relative to unoptimized execution (`-O0`) as a function of 2D grid size. Both `-O2` and `-O3` achieve the highest gains.**

Algorithm 2: Parallel GPU Execution (Host-Side Coordination)

```
Allocate device memory using Struct of Arrays (SoA) layout;
Copy initial conditions from host to device;
for t = 0 to T_end do
    Launch compute_dt reduction kernel;
    Launch compute_k_grid kernel ⟪⟪Grid, Block⟫⟫;
    Launch compute_fluxes kernel ⟪⟪Grid, Block⟫⟫;
    Launch update_state kernel ⟪⟪Grid, Block⟫⟫;
end
Copy final results from device to host for visualization;
```

$$S_y = g_y h - \frac{v}{|\boldsymbol{u}|} h g_z \mu - h k \operatorname{sgn}\left(\frac{\partial v}{\partial x}\right) \sin\phi \frac{\partial (g_z h)}{\partial x}, \tag{14}$$

where $|\boldsymbol{u}| = \sqrt{u^2 + v^2}$ is the local flow speed (regularized by $\epsilon_r = 10^{-3}$ to prevent division by zero) and $\mu = \tan\delta$ is the basal friction coefficient. The first term represents gravitational body forcing, the second models Coulomb friction opposing the local velocity, and the third couples the two momentum equations through the Mohr–Coulomb internal cross-stress.

B. Dynamic Earth-Pressure Coefficients

In two-dimensional flow, different parts of the granular body may simultaneously be in an active (extending) or passive (compressing) stress state. The two states admit distinct coefficients derived from the Mohr–Coulomb yield criterion:

$$k = \frac{2}{\cos^2\phi}\left(1 \mp \sqrt{1 - \cos^2\phi\left(1 + \tan^2\delta\right)}\right) - 1, \tag{15}$$

where the minus sign gives k_{act} (active, material extending) and the plus sign gives k_{pass} (passive, material compressing). For $\phi = 37.3°$ and $\delta = 27.0°$, one obtains $k_{\text{act}} < 1 < k_{\text{pass}}$.

1) State Selection via Velocity Divergence: The local stress state is identified at each grid cell from the sign of

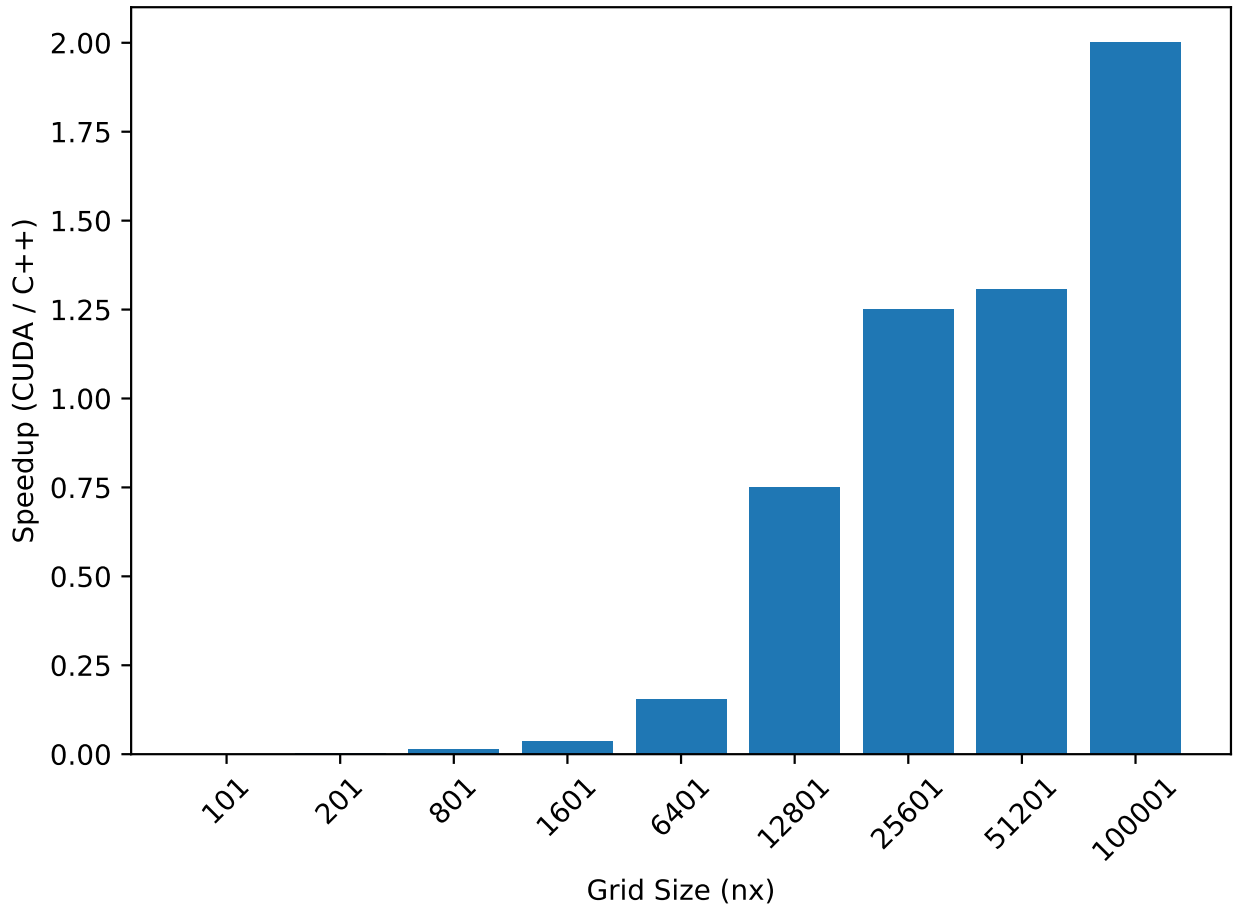

Figure 6: **Speedup of GPU (CUDA) relative to the optimized C++ implementation as a function of 1D grid size N_x. Values greater than 1 indicate GPU outperforming CPU; the crossover occurs near $N_x \approx 25{,}601$.**

the depth-averaged velocity divergence:

$$\nabla \cdot \bar{\boldsymbol{u}} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}. \tag{16}$$

Expanding flow ($\nabla \cdot \bar{\boldsymbol{u}} \geq 0$) selects $k = k_{\text{act}}$; compressing flow selects $k = k_{\text{pass}}$. Spatial derivatives are computed using central differences at interior nodes and zero-gradient conditions at boundaries. The field $k(x, y, t)$ is updated every time step so the pressure response evolves dynamically.

C. Finite-Volume Rusanov Scheme

The governing system (10)–(12) is written in compact conservative form

$$\frac{\partial \boldsymbol{U}}{\partial t} + \frac{\partial \boldsymbol{F}(\boldsymbol{U})}{\partial x} + \frac{\partial \boldsymbol{G}(\boldsymbol{U})}{\partial y} = \boldsymbol{S}(\boldsymbol{U}), \tag{17}$$

with state vector $\boldsymbol{U} = (h,\, hu,\, hv)^\top$ and flux vectors

$$\boldsymbol{F} = \begin{pmatrix} hu \\ hu^2 + \frac{1}{2} k g_z h^2 \\ huv \end{pmatrix}, \quad \boldsymbol{G} = \begin{pmatrix} hv \\ huv \\ hv^2 + \frac{1}{2} k g_z h^2 \end{pmatrix}. \tag{18}$$

1) Rusanov (Local Lax–Friedrichs) Interface Flux: At each cell interface, the Rusanov numerical flux is

$$\hat{\boldsymbol{F}}_{i+1/2,j} = \tfrac{1}{2}\big(\boldsymbol{F}_L + \boldsymbol{F}_R\big) - \tfrac{1}{2}\, a_{i+1/2,j}\big(\boldsymbol{U}_R - \boldsymbol{U}_L\big), \tag{19}$$

where the local maximum wave speed is

$$a_{i+1/2,j} = \max\big(|u_L| + c_L,\ |u_R| + c_R\big), \qquad c = \sqrt{k g_z h}. \tag{20}$$

An analogous expression applies to $\hat{\boldsymbol{G}}_{i,j+1/2}$ with v replacing u. The Rusanov scheme is first-order accurate and provides strong stability guarantees without requiring a full Riemann solver, making it particularly suitable for flows with dry fronts where $h \to 0$.

2) Dry-Bed Treatment: Cells with $h < h_{\text{dry}} = 10^{-4}\,\text{m}$ are treated as dry. Both momentum components are set to zero in such cells, preventing physically spurious velocity reconstruction near the avalanche margins.

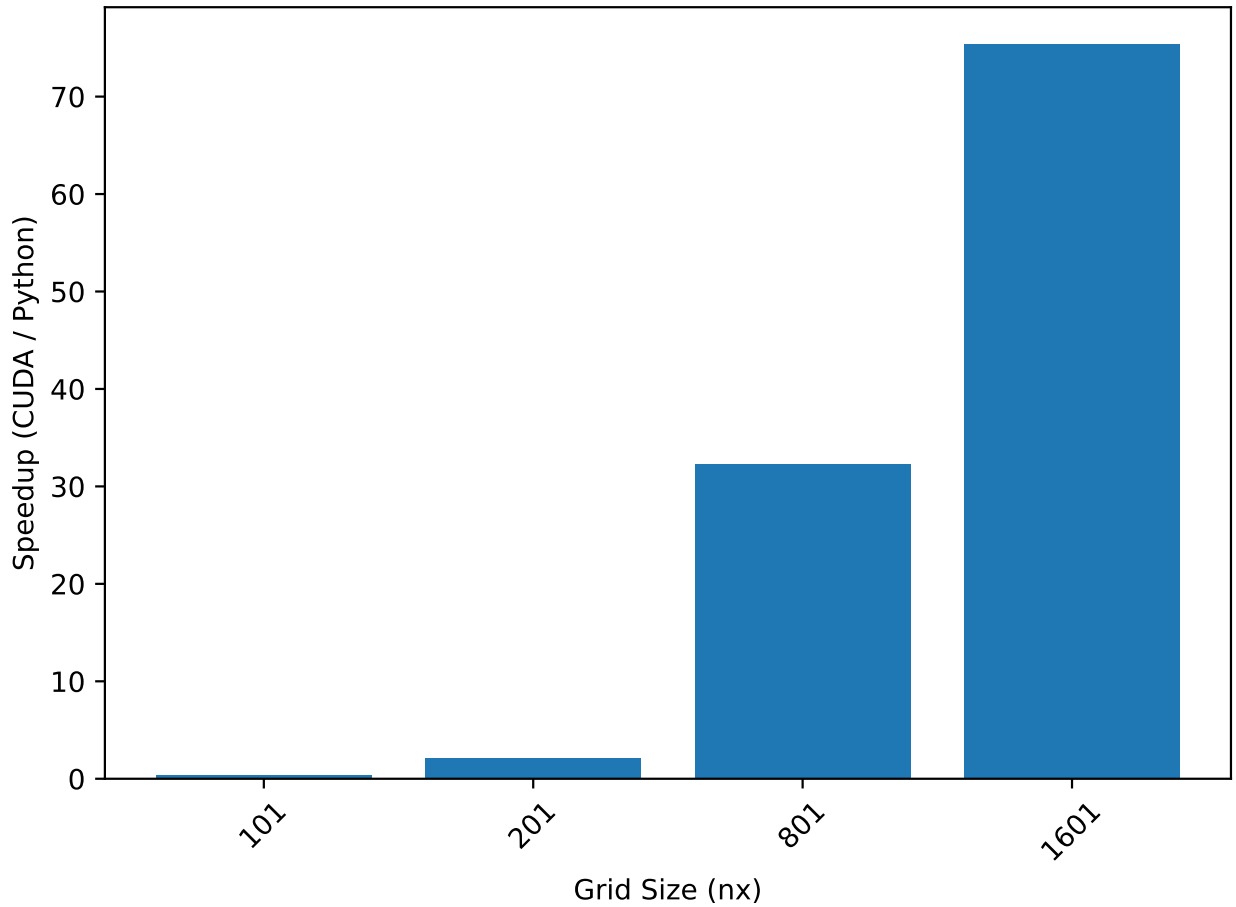

Figure 7: **Speedup of GPU (CUDA) relative to the Python (NumPy) implementation as a function of 1D grid size N_x. The GPU achieves more than $70\times$ speedup at $N_x = 1601$.**

3) Time Integration and CFL Condition: An explicit forward-Euler scheme advances the solution in time:

$$\begin{aligned} \boldsymbol{U}_{i,j}^{n+1} = \boldsymbol{U}_{i,j}^{n} &- \frac{\Delta t}{\Delta x}\big(\hat{\boldsymbol{F}}_{i+1/2,j} - \hat{\boldsymbol{F}}_{i-1/2,j}\big) \\ &- \frac{\Delta t}{\Delta y}\big(\hat{\boldsymbol{G}}_{i,j+1/2} - \hat{\boldsymbol{G}}_{i,j-1/2}\big) + \Delta t\, \boldsymbol{S}_{i,j}^{n}. \end{aligned} \tag{21}$$

The adaptive time step is determined by the two-dimensional CFL condition:

$$\Delta t = \mathrm{CFL} \times \min\left(\frac{\Delta x}{\max(|u| + c)}, \frac{\Delta y}{\max(|v| + c)}\right), \tag{22}$$

with $\mathrm{CFL} = 0.5$. A reduced CFL is used because $k_{\text{pass}} > k_{\text{act}}$ increases the maximum pressure-wave speed.

X. Two-Dimensional Simulation Setup and TITAN2D Validation

A. Physical Parameters and Initial Condition

The simulated configuration reproduces a hemispherical cap of granular material released on a planar slope inclined at $\zeta = 37.4^\circ$. Physical and numerical parameters are listed in Table 1.

Table 1: **Physical and numerical parameters for the 2-D simulation.**

Parameter	Symbol	Value	Units
Slope angle	ζ	37.4	$^\circ$
Basal friction	δ	27.0	$^\circ$
Internal friction	ϕ	37.3	$^\circ$
Active coefficient	k_{act}	computed	—
Passive coefficient	k_{pass}	computed	—
Domain (x)	L_x	4.0	m
Domain (y)	L_y	1.3	m
Grid resolution	$N_x \times N_y$	800×260	—
CFL number	—	0.5	—
Dry threshold	h_{dry}	10^{-4}	m

The granular pile is initialized as a hemisphere of radius $R = 0.06\,\mathrm{m}$ centered at $(x_c, y_c) = (0.4, 0.65)\,\mathrm{m}$, with

maximum height $H_0 = R/1.5$:

$$h(x, y, 0) = \begin{cases} H_0 & (x - x_c)^2 + (y - y_c)^2 \leq R^2, \\ 0 & \text{otherwise.} \end{cases} \tag{23}$$

The initial velocity field is zero everywhere. As time advances, gravitational forcing accelerates the mass downslope, the pile spreads laterally due to internal pressure gradients, and Coulomb basal friction progressively decelerates the flow.

B. Centerline Profile Validation

TITAN2D is an open-source, parallel finite-volume solver developed specifically for granular avalanche simulations [1]. It solves the same depth-averaged Mohr–Coulomb system on an adaptive mesh using a Godunov-type scheme and constitutes a well-established numerical reference. Centerline profiles of $h(x, y_c, t)$ are extracted at $y = y_c = 0.65\,\text{m}$ and compared at five instants: $t = 0$, 0.5, 1.0, 1.5, 2.0 s (Fig. 8).

C. **Spatial and Temporal Convergence**

To ensure that the numerical results are robust and independent of the chosen discretization parameters, an asymptotic self-convergence study was conducted. Evaluating global error norms (such as L_2) between schemes of different spatial orders can artificially inflate error metrics due to minor phase lags at the shock front. To bypass this, the convergence study tracked a single scalar value: the avalanche depth h at a specific Eulerian probe location ($x = 1.0\,\text{m}, y = 0.65\,\text{m}$) at a fixed time ($t = 1.0\,\text{s}$).

Spatial convergence was evaluated by systematically refining the grid resolution from $N_x = 100$ up to $N_x = 1600$ while maintaining a constant aspect ratio and a fixed Courant number (CFL $= 0.5$). Figure 9 illustrates the measured avalanche height at the probe as a function of grid density. As the grid is refined ($\Delta x \to 0$), the truncation error of the numerical scheme diminishes, and the solution approaches a stable asymptote. The relative percentage change in depth between successive refinements drops sharply, falling to just 2.19% between $N_x = 1200$ and $N_x = 1600$. This analysis mathematically proves diminishing returns on extreme spatial refinement, identifying intermediate grids (such as 400×130 to 800×260) as the optimal balance between high-fidelity shock capture and computational efficiency.

Temporal convergence was verified by holding the spatial grid constant (400×130) and systematically reducing the time step size (Δt) by lowering the CFL number from 0.8 down to 0.05. As shown in Figure 10, the measured depth remains virtually constant across the entire stable Courant envelope. Because the numerical formulation utilizes an explicit scheme, spatial discretization errors (inherent to the first-order Rusanov flux) heavily dominate over time-integration errors. The flat temporal convergence curve confirms that decreasing the time step further does not yield additional physical accuracy. Consequently, the solver can be safely operated near its maximum stable limit (CFL $= 0.5$) to maximize GPU processing throughput without compromising the fidelity of the simulation.

D. Spatial Footprint and Error Metrics (IoU & RMSE)

The numerical footprint is compared against TITAN2D using a wet-cell threshold $h_{\text{thresh}} = 10^{-3}\,\text{m}$. Geometric alignment is evaluated using the Intersection-over-Union (IoU) metric:

$$\text{IoU}(t) = \frac{\text{Area}_{\text{Rusanov}} \cap \text{Area}_{\text{TITAN2D}}}{\text{Area}_{\text{Rusanov}} \cup \text{Area}_{\text{TITAN2D}}}, \tag{24}$$

and the Root-Mean-Square Error (RMSE) of flow depth is computed exclusively over wetted regions. Figure 11 illustrates the temporal evolution of both metrics, demonstrating strong footprint fidelity and bounded depth error throughout the primary acceleration phase. The cumulative wetted footprint is shown in Fig. 12.

E. Macroscopic Flow Dynamics ($h_{\max}$ and $\bar{V}$)

The maximum peak height $h_{\max}$ is tracked continuously to measure the vertical collapse of the initial parabolic cap. The kinetic energy evolution is quantified via a mass-weighted average velocity:

$$\bar{V}(t) = \frac{\sum_{i,j} h_{i,j} \sqrt{u_{i,j}^2 + v_{i,j}^2}}{\sum_{i,j} h_{i,j}}, \tag{25}$$

which captures the bulk acceleration of the avalanche under gravitational forcing, followed by frictional deceleration. Results are shown in Fig. 13.

F. Lagrangian Kinematics (Centroid Tracking)

The trajectory of the avalanche center-of-mass is calculated from the depth distribution:

$$X_c(t) = \frac{\sum_{i,j} x_i h_{i,j}}{\sum_{i,j} h_{i,j}}, \qquad Y_c(t) = \frac{\sum_{i,j} y_j h_{i,j}}{\sum_{i,j} h_{i,j}}. \tag{26}$$

Mapping $\bar{V}$ against the downslope centroid displacement $d(t) = \sqrt{(X_c - X_{c,0})^2 + (Y_c - Y_{c,0})^2}$ provides insight into run-out distances and momentum conservation (Fig. 14).

G. Discussion of Results

The validation suite demonstrates strong agreement between the present Dynamic-k Rusanov solver and TITAN2D across multiple physical metrics.

a) Wave Speed and Profile Shape.: Both solvers predict the downslope advance of the avalanche front at nearly identical locations. The parabolic thinning and progressive flattening of the profile are captured correctly. At early times ($t \leq 0.5\,\text{s}$), the Rusanov solution exhibits minor numerical diffusion characteristic of first-order fluxes, which diminishes as the flow transitions to a spreading-dominated regime.

b) Dynamic k Switching.: Near the leading edge, material actively extends ($\nabla \cdot \bar{\boldsymbol{u}} > 0$), selecting k_{act}. Behind the front, decelerating material compresses, triggering the local switch to k_{pass}. This localized stiffening of the pressure wave correctly steepens the trailing profile slope, validating the 2D divergence field implementation.

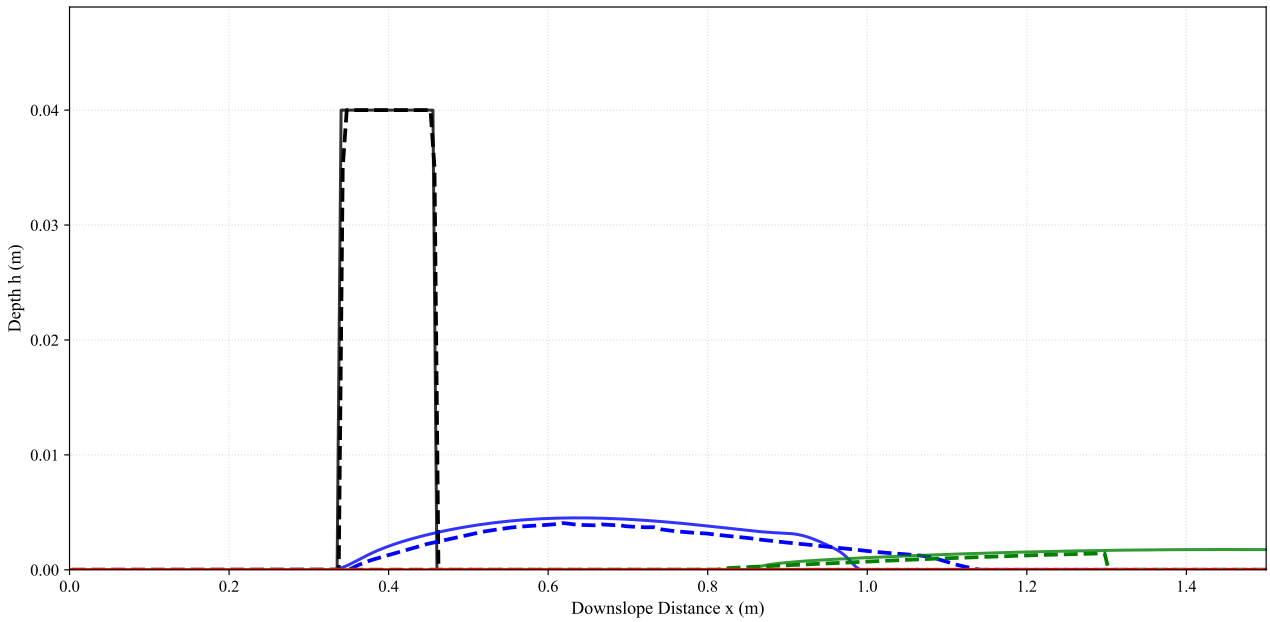

Figure 8: Centerline thickness profiles $h(x, y_c, t)$ at five time instants. Dashed lines: TITAN2D reference; solid lines: present Dynamic-k Rusanov solver. Colors indicate time: $t = 0\,\mathrm{s}$ (black), $t = 0.5\,\mathrm{s}$ (blue), $t = 1.0\,\mathrm{s}$ (green), $t = 1.5\,\mathrm{s}$ (orange), $t = 2.0\,\mathrm{s}$ (red).

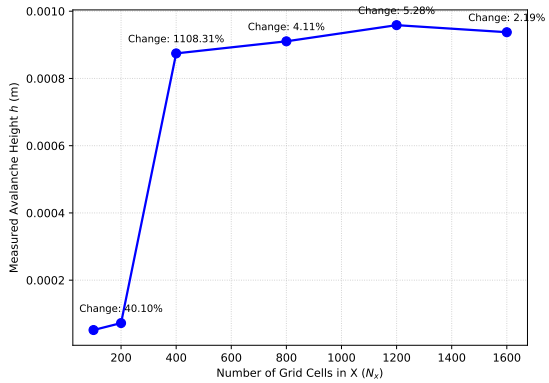

Figure 9: Spatial convergence study demonstrating diminishing returns. The solid line with circular markers represents the measured avalanche depth h (in meters) as a function of the longitudinal grid resolution (N_x). The depth approaches an asymptotic value as the grid is refined, with the relative change dropping below 3% at ultra-fine resolutions.

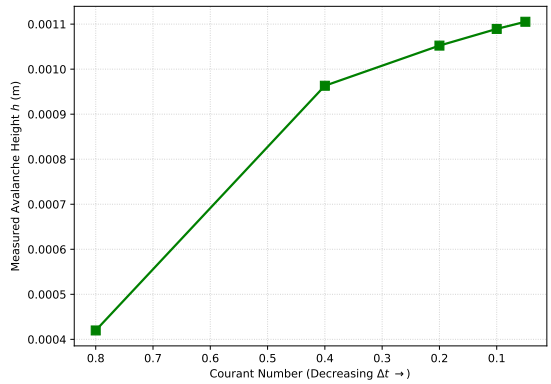

Figure 10: Temporal convergence study. The solid line with square markers displays the measured avalanche depth h (in meters) evaluated across decreasing Courant numbers. The flatline response indicates that the explicit solution is independent of the time step size within the stable Courant limit, proving that spatial error heavily dominates over temporal error.

c) Global Mass Conservation.: Figure 15 confirms that the total volume $\mathcal{V}(t) = \iint h\,dx\,dy$ remains strictly constant throughout the simulation, verifying that the discrete finite-volume update conserves mass to numerical precision.

H. Comparison with the One-Dimensional Model

Comparing the two-dimensional results with the one-dimensional solver reveals several important differences. First, lateral spreading is absent in the 1-D model by construction, whereas the 2-D simulation shows the pile widening in the y-direction at a rate determined by $kg_z\,\partial h/\partial y$, which reduces the peak height more rapidly. Second, the dynamic switching between k_{act} and k_{pass} has a more pronounced effect in 2-D because the divergence field $\nabla \cdot \bar{\boldsymbol{u}}$ captures compression events that occur even when the downslope component alone is expanding, producing a richer spatial structure of the pressure coefficient field. Third, the

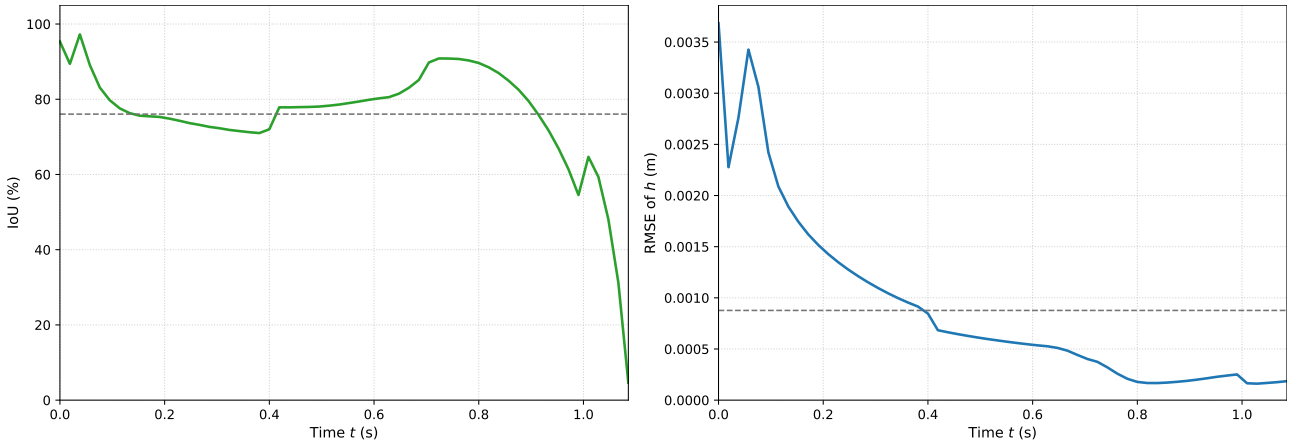

Figure 11: **Quantitative spatial validation. (Left) Intersection-over-Union (IoU) measuring the overlap of the wetted footprint between the Dynamic-k Rusanov solver and TITAN2D. (Right) RMSE of flow depth h computed on strictly wet cells.**

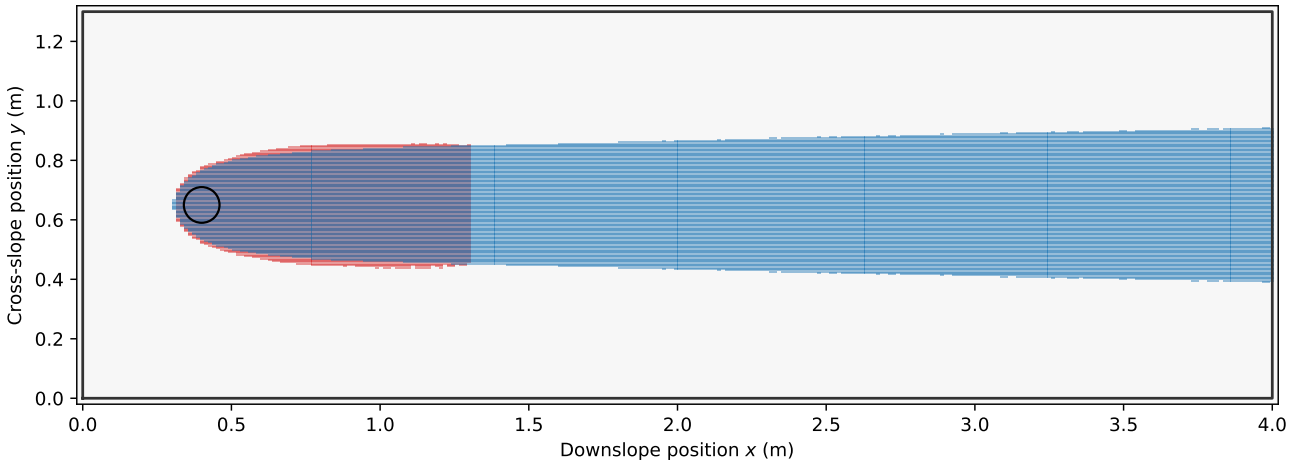

Figure 12: **Cumulative wetted footprint map comparing the maximum lateral and longitudinal spread of the Dynamic-k Rusanov solver (blue) against TITAN2D (red dashed).**

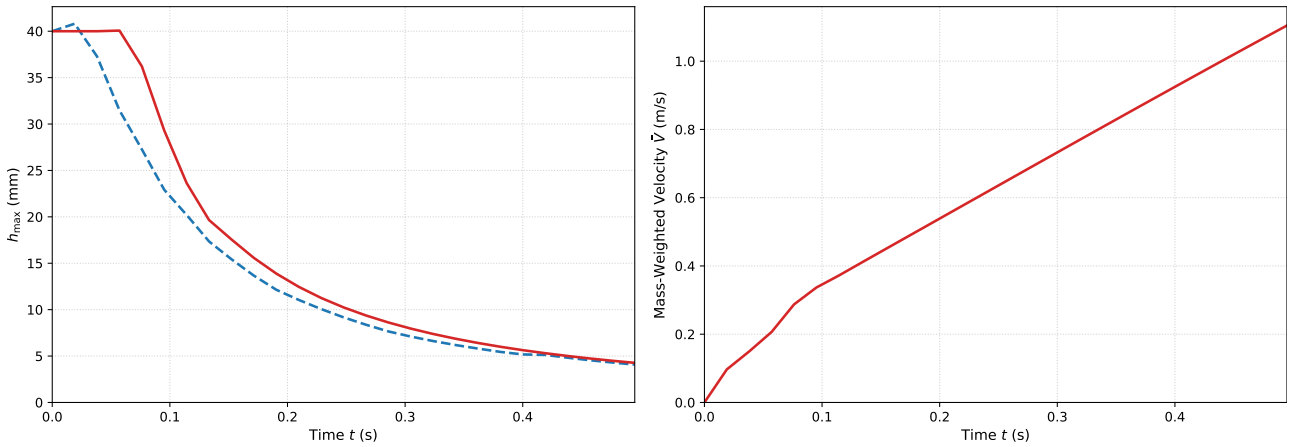

Figure 13: **Macroscopic flow dynamics. (Left) Collapse of maximum flow thickness $h_{\max}(t)$ over time. (Right) Mass-weighted average velocity $\bar{V}(t)$ showing the gravitational acceleration phase followed by frictional deceleration.**

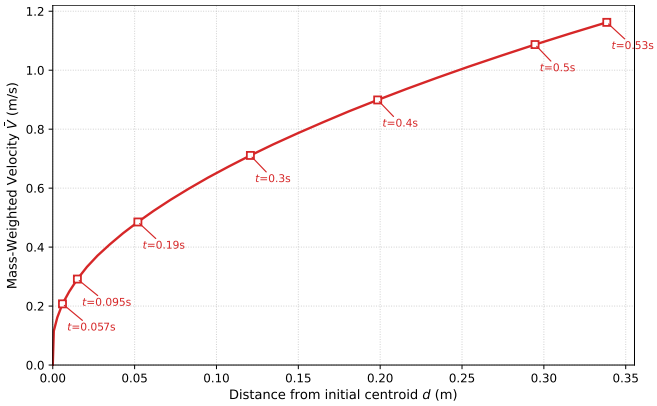

Figure 14: Lagrangian trajectory analysis: mass-weighted velocity $\bar{V}$ as a function of centroid displacement $d(t)$. Selected time stamps are annotated.

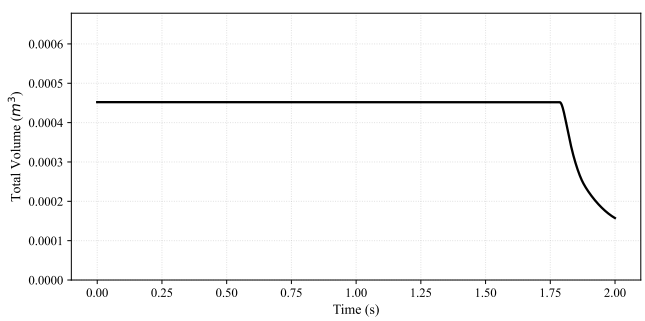

Figure 15: Total granular volume $\mathcal{V}(t)$ as a function of time, confirming exact mass conservation in the Dynamic-k Rusanov simulation.

Rusanov flux scheme replaces the combination of upwind advection and centered pressure gradients of the 1-D finite-difference solver with a unified, conservative numerical flux, ensuring consistent treatment of shocks and dry fronts within a single formulation.

Nomenclature

h	depth-averaged flow thickness (m)
u, v	depth-averaged velocity, x- and y-dir. ($\mathrm{m\,s^{-1}}$)
k	earth-pressure coefficient (–)
k_{act}, k_{pass}	active and passive earth-pressure coeff. (–)
ϕ	internal friction angle (rad)
δ	basal friction angle (rad)
ζ	slope inclination angle (rad)
μ	basal friction coefficient, $\mu = \tan\delta$ (–)
g_x, g_y, g_z	gravity components along x, y, z ($\mathrm{m\,s^{-2}}$)
$\boldsymbol{F}$, $\boldsymbol{G}$	flux vectors in x- and y-directions
$\boldsymbol{S}$	source term vector
Δx, Δy	grid spacing (m)
Δt	time step (s)
h_{dry}	dry-bed threshold (m)

XI. Conclusion

This paper developed a high-fidelity numerical solver for granular avalanche motion modeled by the depth-averaged Savage–Hutter equations. Theoretical validation against analytical 1D similarity solutions confirmed accurate mass preservation and spreading dynamics. The computational framework was then extended to a complete 2D finite-volume architecture employing a Rusanov Riemann solver with dynamic active/passive earth-pressure coefficient switching.

Rigorous quantitative benchmarking against the TITAN2D Godunov-type solver verified the physical accuracy of the 2D model. The validation suite utilized IoU spatial footprint analysis, localized RMSE depth tracking, and Lagrangian centroid transport evaluation, demonstrating excellent agreement in capturing the rapid acceleration, geometric collapse, lateral spreading, and ultimate frictional deceleration of the avalanche mass.

A massively parallel GPU architecture was engineered using CUDA to circumvent the $\mathcal{O}(N_x \times N_y)$ sequential bottleneck inherent to high-resolution Eulerian grids. Transitioning state vectors to a Struct of Arrays (SoA) layout and mapping explicit interface computations directly to parallel hardware threads allowed the solver to fully saturate GDDR memory bandwidth. Performance evaluations revealed that the optimized CUDA implementation achieved up to $70\times$ speedup over interpreted Python and drastically outperformed C++ for large domain topographies, confirming that GPU parallelization is a critical enabling technology for real-time, high-resolution geophysical hazard assessment.

References

[1] Patra, A. K., Bauer, A. C., Nichita, C. C., Pitman, E. B., Sheridan, M. F., Bursik, M., Rupp, B., Webber, A., Stinton, A. J., Namikawa, L. M., & Renschler, C. S. (2005). *Parallel adaptive numerical simulation of dry avalanches over natural terrain*. *Journal of Volcanology and Geothermal Research*, 139(1–2), 1–21.